Norges miljø- og biovitenskapelige universitet

# TEL200 – Introduction to Robotics

**David A. Anisi**

Chapter 8: Manipulator velocity

# Agenda

8.1 Manipulator Jacobian

8.3 Jacobian Condition and Manipulability

20-DOF snake-robot arm
Photo courtesy: OC Robotics

# Introduction

- A robot's end-effector moves in ***Cartesian space*** with a translational and rotational velocity – a spatial velocity;
- However, that ***velocity*** is a consequence of the velocities of the individual robot joints;
- This section introduces the ***relationship*** between the velocity of the joints $\dot{q}$ and the spatial velocity $v$ of the end-effector: the ***Jacobian matrix***.

# Introduction

- The ***Jacobian matrix*** can be used to compute:
  - ❑ Manipulability (Sec 8.1, 8.3);
  - ❑ Resolved-rate motion control (Sec 8.2);
  - ❑ Force-velocity Relationships (Sec 8.4);
  - ❑ Numerical Inverse Kinematics (Sec 8.5)

  …among others.

Not included in TEL200

**Carl Gustav Jacob Jacobi (1804–1851)** was a Prussian mathematician. Jacobi wrote a classic treatise on elliptic functions in 1829 and also described the derivative of $m$ functions of $n$ variables which bears his name.

# Introduction – Jacobian matrix

- A Jacobian is the matrix equivalent of the derivative – the derivative of a vector-valued function of a vector with respect to a vector.
- If $f : R^n \rightarrow R^m$, such that each of its first-order partial derivatives exists on R, then Jacobian matrix of **f**, denoted $\mathbf{J}_f \in \mathbf{R}^{m \times n}$, is defined as

$$\mathbf{J}_{\mathbf{f}} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x_1} & \cdots & \frac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \nabla^{\mathsf{T}} f_1 \\ \vdots \\ \nabla^{\mathsf{T}} f_m \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

# Manipulator Jacobian

- In the last chapter we discussed the relationship between ***joint coordinates*** and ***end-effector pose*** – the manipulator kinematics;

$$\boldsymbol{\xi}_{\boldsymbol{E}} = \mathcal{K}(\boldsymbol{q}) \tag{1}$$

# Manipulator Jacobian

- In the last chapter we discussed the relationship between ***joint coordinates*** and ***end-effector pose*** – the manipulator kinematics;

$$\boldsymbol{\xi}_{\boldsymbol{E}} = \mathcal{K}(\boldsymbol{q}) \tag{1}$$

- Now, we investigate the relationship between the ***rate*** of change of these quantities – between ***joint velocity*** and ***velocity*** of the end-effector;

$$\dot{\boldsymbol{\xi}}_{\boldsymbol{E}} = J(\boldsymbol{q})\,\dot{\boldsymbol{q}} \tag{2}$$

- This is called the velocity or ***differential kinematics*** of the manipulator.

# Overview of Ch. 7-8

Forward

$\mathbf{q}$ **Kinematics** $\xi$

Backward

Joint space ⟷ Task space

Matrix multiplication, J(q)

$\dot{q}$ **Jacobian** $\dot{\xi}$

Inverse, $J^{-1}(q)$

Joint space velocity ⟷ Task space velocity

# Overview of Ch. 7-8

Forward

$\mathbf{q}$ **Kinematics** $\xi$

Backward

Joint space ⟷ Task space

Matrix multiplication, J(q)

$\dot{q}$ **Jacobian** $\dot{\xi}$

Inverse, $J^{-1}(q)$

Joint space velocity ⟷ Task space velocity

- Jacobian Matrix: Relationship between joint space velocity with task space velocity
- Physical Interpretation: How each individual joint space velocity contribute to task space velocity
- Question: When is the **Jacobian not invertible**? Answer: **at robot singularities**

# Jacobian in the World Coordinate Frame

- We illustrate the basics with our now familiar 2-dimensional example (Fig. 8.1), this time defined using ***Denavit-Hartenberg notation***;

```
>>> import sympy

>>> a1, a2 = sympy.symbols("a1, a2")

>>> e = ERobot2(ET2.R() * ET2.tx(a1) * ET2.R() *
ET2.tx(a2))
```

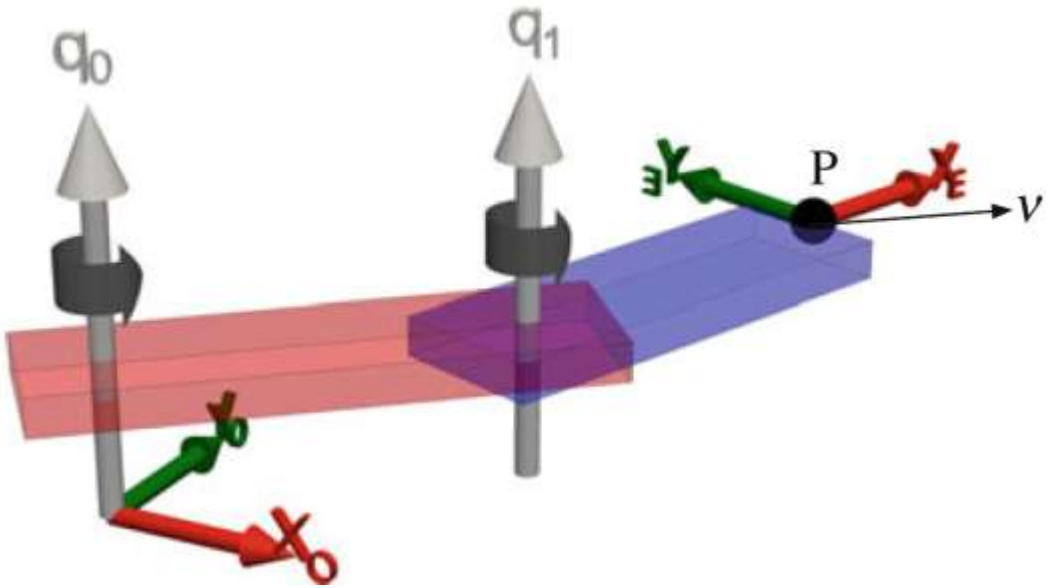



**Fig.8.1**:Two-link robot showing the end-effector position $p = (x, y)$ and the Cartesian velocity vector $v = dp/dt$.

# Jacobian in the World Coordinate Frame

- Define a symbolic 2-vector to represent the joint angles

```
>>> q = sympy.symbols("q:2")
(q0, q1)
```

- The forward kinematics are

```
>>> TE = e.fkine(q);
```

The position vector of the robot end-effector $p = (x, y) \in \mathbb{R}^2$ is given by

```
>>> p = TE.t
array([a1*cos(q0) + a2*cos(q0 + q1),
a1*sin(q0) + a2*sin(q0 + q1)], dtype=object)
```

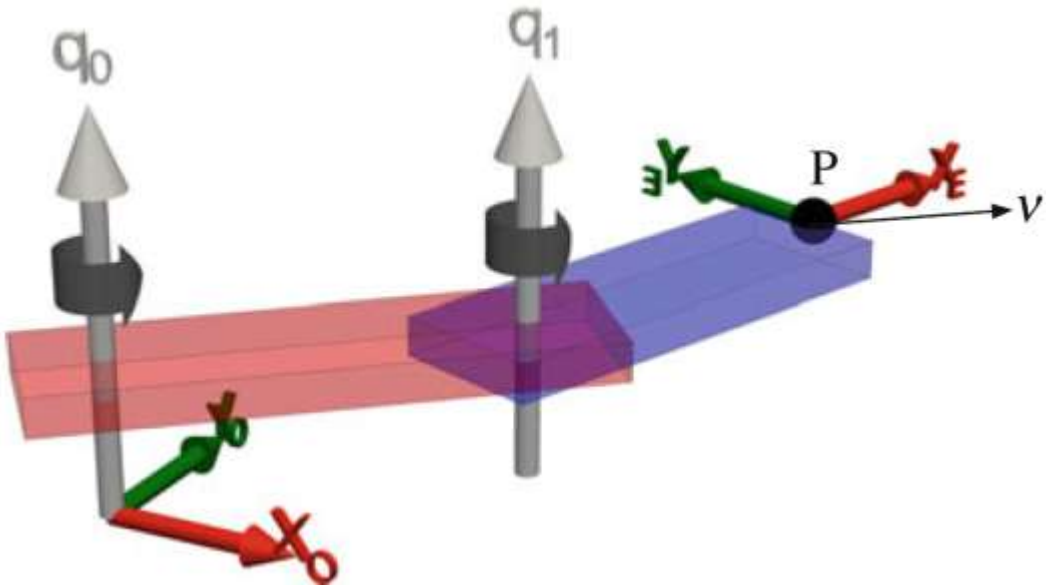



**Fig.8.1**:Two-link robot showing the end-effector position $p = (x, y)$ and the Cartesian velocity vector $v = dp/dt$.

# Jacobian in the World Coordinate Frame

…and we compute the ***time-derivative*** of $p$ with respect to the joint variables $q$.

- Since $p$ and $q$ are both vectors the derivative:

$$\frac{\mathrm{d}\boldsymbol{p}}{\mathrm{d}\boldsymbol{q}} = \boldsymbol{J}(\boldsymbol{q}) \qquad (8.1)$$

will be a matrix – a ***Jacobian*** matrix:

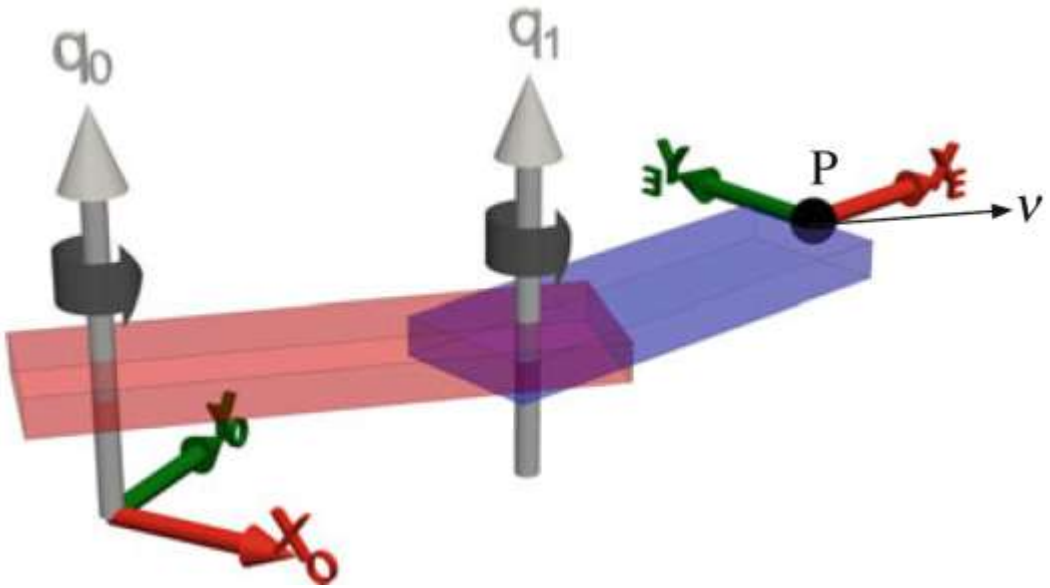



**Fig.8.1**:Two-link robot showing the end-effector position $p = (x, y)$ and the Cartesian velocity vector $v = dp/dt$.

# Jacobian in the World Coordinate Frame

…and we compute the ***time-derivative*** of $p$ with respect to the joint variables $q$.

- Since $p$ and $q$ are both vectors the derivative:

$$\frac{\mathrm{d}\boldsymbol{p}}{\mathrm{d}\boldsymbol{q}} = \boldsymbol{J}(\boldsymbol{q}) \qquad (8.1)$$

will be a matrix – a ***Jacobian*** matrix:

```
>>> J = sympy.Matrix(p).jacobian(q)
Matrix([[-a1*sin(q0) - a2*sin(q0 + q1), -a2*sin(q0 + q1)],[ a1*cos(q0) + a2*cos(q0 + q1), a2*cos(q0 + q1)]])
>>> J.shape
(2, 2)
```

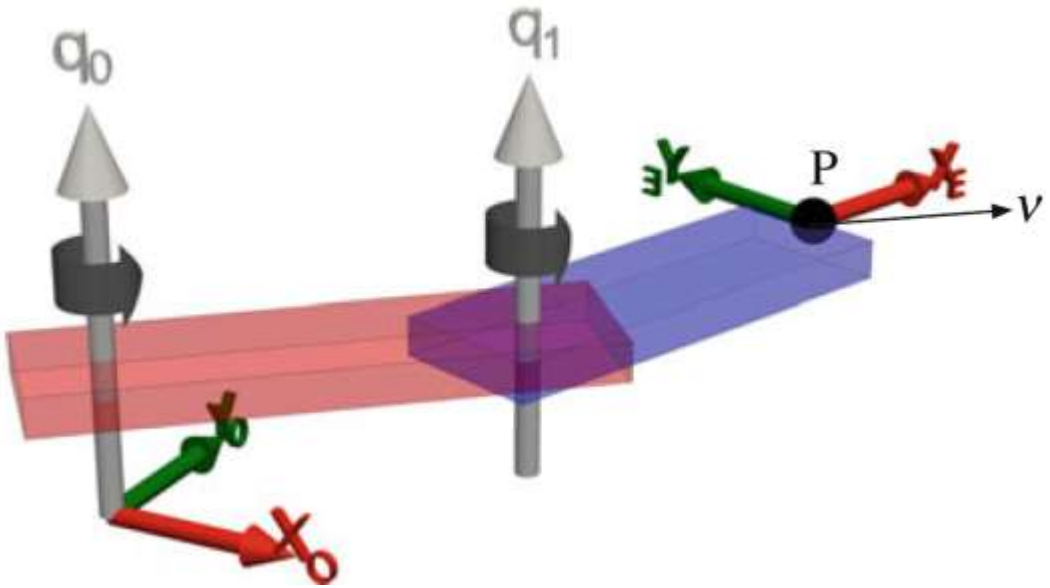



**Fig.8.1**:Two-link robot showing the end-effector position $p = (x, y)$ and the Cartesian velocity vector $v = dp/dt$.

# Jacobian in the World Coordinate Frame

- To determine the relationship between ***joint velocity*** and *end-effector velocity* we rearrange Eq. (8.1) as:

$$\frac{\mathrm{d}\boldsymbol{p}}{\mathrm{d}\boldsymbol{q}} = \boldsymbol{J}(\boldsymbol{q}) \qquad \Rightarrow \qquad \boxed{\mathrm{d}\boldsymbol{p} = \boldsymbol{J}(\boldsymbol{q})\mathrm{d}\boldsymbol{q}} \tag{4}$$

# Jacobian in the World Coordinate Frame

- To determine the relationship between ***joint velocity*** and *end-effector velocity* we rearrange Eq. (8.1) as:

$$\frac{\mathrm{d}\boldsymbol{p}}{\mathrm{d}\boldsymbol{q}} = J(\boldsymbol{q}) \qquad \Rightarrow \qquad \boxed{\mathrm{d}\boldsymbol{p} = J(\boldsymbol{q})\mathrm{d}\boldsymbol{q}} \tag{4}$$

and ***divide*** through by $dt$ to obtain:

$$\frac{\mathrm{d}\boldsymbol{p}}{\mathrm{d}t} = J(\boldsymbol{q})\frac{\mathrm{d}\boldsymbol{q}}{\mathrm{d}t} \qquad \Rightarrow \qquad \boxed{\dot{\boldsymbol{p}} = J(\boldsymbol{q})\dot{\boldsymbol{q}}} \tag{5}$$

# Jacobian in the World Coordinate Frame

- To determine the relationship between ***joint velocity*** and *end-effector velocity* we rearrange Eq. (8.1) as:

$$\frac{\mathrm{d}\boldsymbol{p}}{\mathrm{d}\boldsymbol{q}} = J(\boldsymbol{q}) \qquad \Rightarrow \qquad \boxed{\mathrm{d}\boldsymbol{p} = J(\boldsymbol{q})\mathrm{d}\boldsymbol{q}} \tag{4}$$

and ***divide*** through by $dt$ to obtain:

$$\frac{\mathrm{d}\boldsymbol{p}}{\mathrm{d}t} = J(\boldsymbol{q})\frac{\mathrm{d}\boldsymbol{q}}{\mathrm{d}t} \qquad \Rightarrow \qquad \boxed{\dot{\boldsymbol{p}} = J(\boldsymbol{q})\dot{\boldsymbol{q}}} \tag{5}$$

- The Jacobian matrix ***maps*** velocity from the joint coordinate or configuration space to the end-effector's coordinate or Cartesian space and is itself a function of the joint coordinates.

# Jacobian in the World Coordinate Frame

- More generally we write the ***forward kinematics*** in functional form as:

$$^{0}\xi = \mathcal{K}(\boldsymbol{q})$$

# Jacobian in the World Coordinate Frame

- More generally we write the ***forward kinematics*** in functional form as:

$$^{0}\xi = \mathcal{K}(\boldsymbol{q})$$

and taking the ***time-derivative*** we write

$$^{0}\boldsymbol{v} = {}^{0}\mathbf{J}(\boldsymbol{q})\,\dot{\boldsymbol{q}} \in \mathbb{R}^{M} \tag{8.2}$$

where $^{0}v = \left(v_x, v_y, v_z, \omega_x, \omega_y, \omega_z\right) \in \mathbb{R}^6$ is the ***spatial velocity*** of the end-effector in the world frame and comprises ***translational*** and ***rotational*** velocity components.

# Jacobian in the World Coordinate Frame

- More generally we write the ***forward kinematics*** in functional form as:

$$^{0}\xi = \mathcal{K}(\boldsymbol{q})$$

and taking the ***time-derivative*** we write

$$^{0}\boldsymbol{v} = {}^{0}\mathbf{J}(\boldsymbol{q})\,\dot{\boldsymbol{q}} \in \mathbb{R}^{M} \tag{8.2}$$

The matrix $^{0}J = (q) \in \mathbb{R}^{6\times N}$ is the manipulator Jacobian.

This relationship is sometimes referred to as the ***instantaneous*** forward kinematics.

# Jacobian in the World Coordinate Frame

- For a real 3-dimensional robot manipulator this ***Jacobian matrix*** can be numerically computed by the `jacob0` method of any Toolbox robot object, based on its Denavit-Hartenberg (DH) parameters.

- For the ***UR5*** robot in the pose shown in Fig. 8.2, the Jacobian is

```
>>> ur5 = models.URDF.UR5();

>>> J = ur5.jacob0(ur5.q1)
```

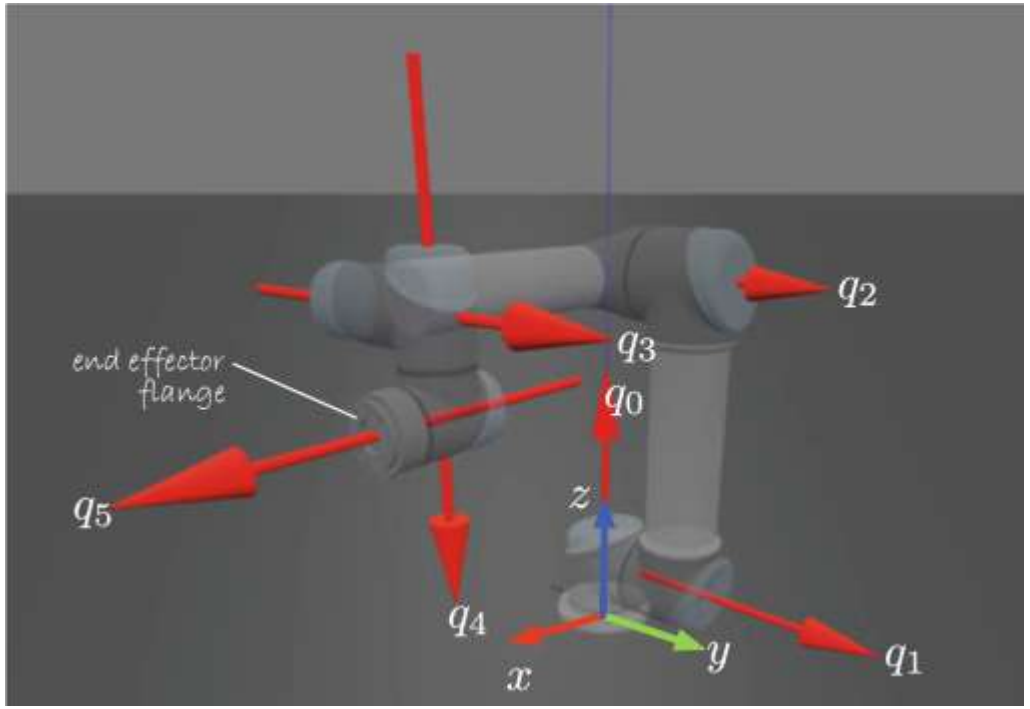



**Fig.8.2:** UR5 robot in joint configuration $q_1$.The end-effector $z$-axis points in the world $x$-direction.

# Jacobian in the World Coordinate Frame

```
J =
array([[-0.02685, 0.3304, -0.09465, -0.09465, -0.0823, 0],
       [ 0.4745,  0,       0,        0,       -0.0823, 0],
       [ 0,      -0.4745, -0.4745,  -0.0823,   0,     -0.0823],
       [ 0,       0,       0,        0,        0,      1],
       [ 0,       1,       1,        1,        0,      0],
       [ 1,       0,       0,        0,       -1,      0]])
```

$J$ is a matrix with dimensions $\dim \mathcal{T} \times \dim \mathcal{C}$ – in this case $6 \times 6$;

- Each ***row*** corresponds to a Cartesian degree of freedom, while each ***column*** corresponds to a joint degree of mobility.

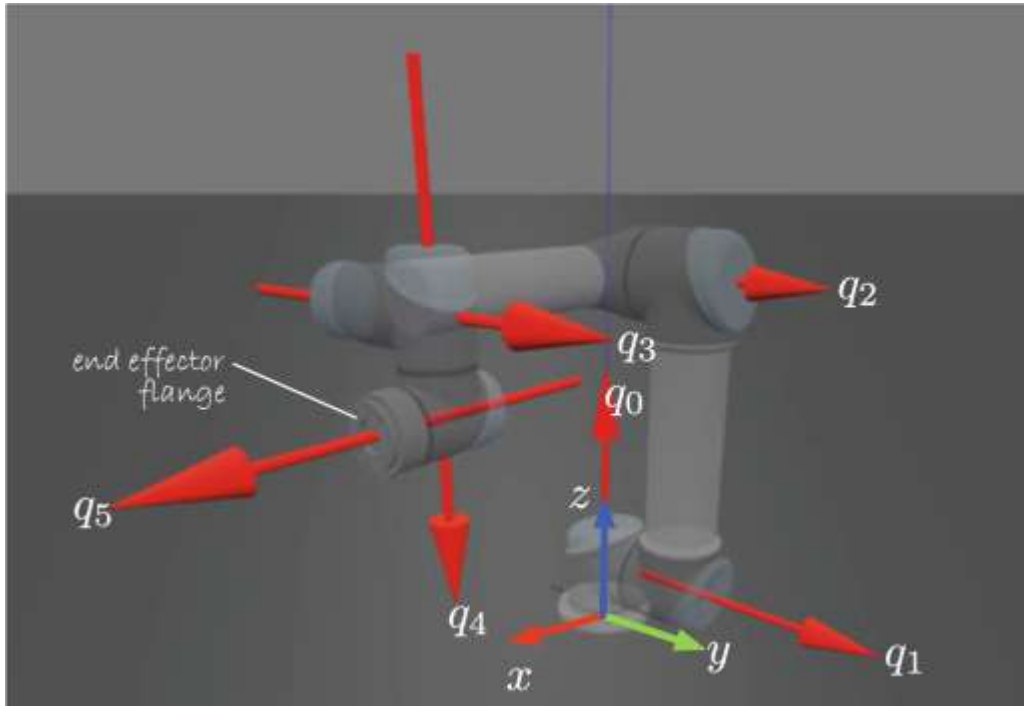



**Fig.8.2:** UR5 robot in joint configuration $q_1$.The end-effector $z$-axis points in the world $x$-direction.

# Jacobian in the World Coordinate Frame

```
J =
array([[-0.02685, 0.3304, -0.09465, -0.09465, -0.0823, 0],
       [ 0.4745,  0,       0,        0,        -0.0823, 0],
       [ 0,      -0.4745, -0.4745,  -0.0823,   0,      -0.0823],
       [ 0,       0,       0,        0,         0,       1],
       [ 0,       1,       1,        1,         0,       0],
       [ 1,       0,       0,        0,        -1,       0]])
```

$J$ is a matrix with dimensions $\dim \mathcal{T} \times \dim \mathcal{C}$ – in this case $6 \times 6$;

- Each ***row*** corresponds to a Cartesian degree of freedom, while each ***column*** corresponds to a joint degree of mobility.

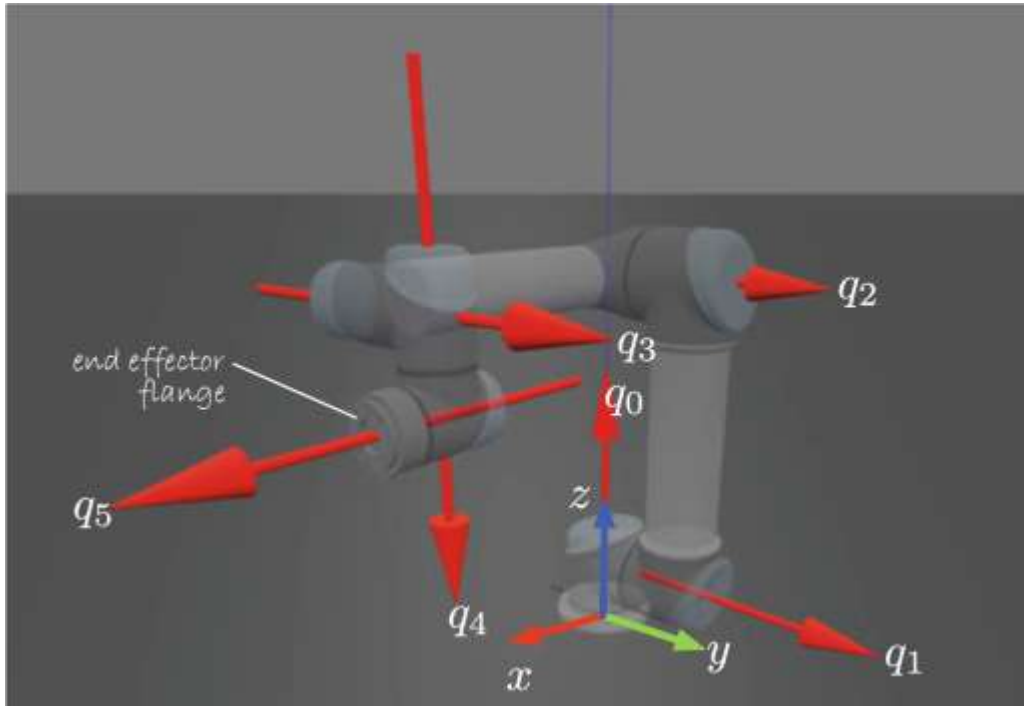



**Fig.8.2:** UR5 robot in joint configuration $q_1$.The end-effector $z$-axis points in the world $x$-direction.

# Jacobian in the End-effector Coordinate Frame

- The ***Jacobian matrix*** computed by the method `jacob0` maps joint velocity to the end-effector spatial velocity expressed in the ***world coordinate*** frame;

# Jacobian in the End-effector Coordinate Frame

- The ***Jacobian matrix*** computed by the method jacob0 maps joint velocity to the end-effector spatial velocity expressed in the ***world coordinate*** frame;
- To obtain the spatial velocity in the ***end-effector coordinate*** frame we introduce the ***velocity transformation*** from the world frame $\{0\}$ to the end-effector frame $\{E\}$ which is a function of the end-effector pose (Ref. Sec 3.1.3):

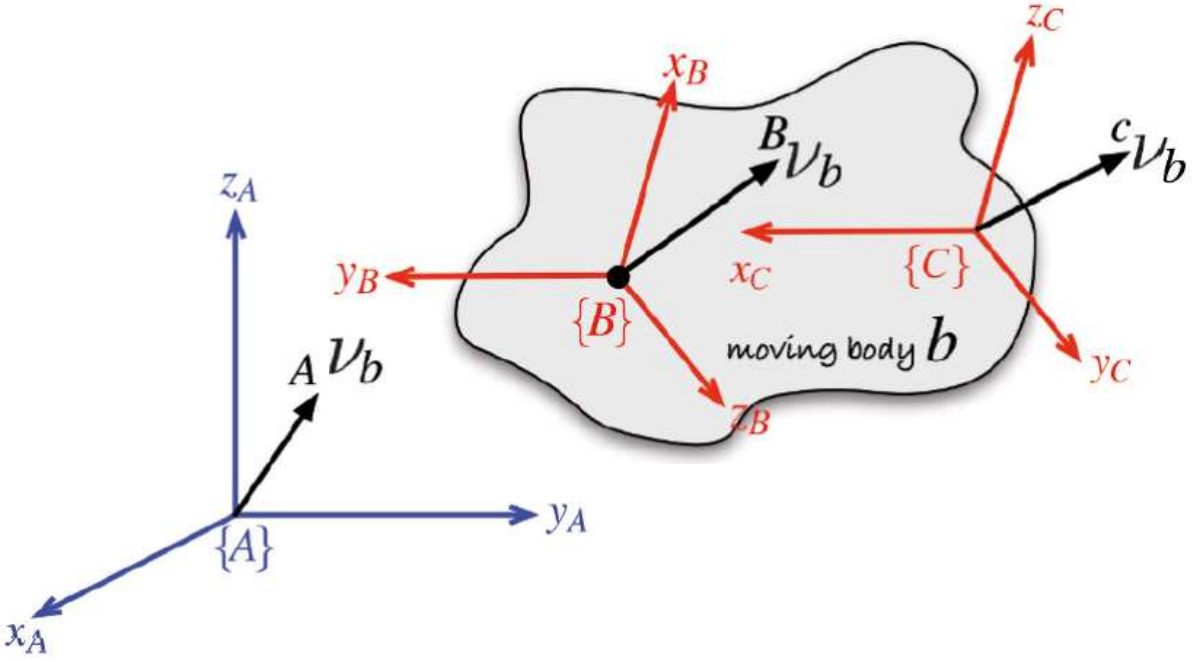
# Transforming Spatial Velocities - from Sec. 3.1.3

- The ***velocity*** of a moving body can be expressed with respect to a world reference frame $\{A\}$ or the moving body frame $\{B\}$
- Spatial velocities are linearly related by:

$$ {}^{A}\nu = \begin{pmatrix} {}^{A}\boldsymbol{R}_B & \mathbf{0}_{3\times3} \\ \mathbf{0}_{3\times3} & {}^{A}\boldsymbol{R}_B \end{pmatrix} {}^{B}\nu = {}^{A}\boldsymbol{J}_B\left({}^{A}\xi_B\right){}^{B}\nu \qquad (3.5) $$

where $\boxed{{}^{A}\xi_B \sim ({}^{A}\boldsymbol{R}_B, {}^{A}\boldsymbol{t}_B)}$ and $\boxed{{}^{A}\boldsymbol{J}_B(\cdot)}$ is a Jacobian or interaction matrix.

Representing the spatial velocity of a moving body $b$ with respect to various coordinate frames. Note that $\nu$ is a 6-dimensional vector



# Jacobian in the End-effector Coordinate Frame

- Using Eq. (8.2)

$$ {}^{0}\boldsymbol{\nu} = {}^{0}\mathbf{J}(\boldsymbol{q})\,\dot{\boldsymbol{q}} \in \mathbb{R}^{M} \tag{8.2} $$

we obtain

$$ {}^{E}\nu = {}^{E}J_0\left({}^{E}\xi_0\right){}^{0}J(q)\dot{q} = \underbrace{\begin{pmatrix} {}^{E}\boldsymbol{R}_0 & 0_{3\times3} \\ 0_{3\times3} & {}^{E}\boldsymbol{R}_0 \end{pmatrix}{}^{0}J(q)}_{{}^{E}J(q)}\,\dot{q} = {}^{E}J(q)\dot{q} \tag{8.3} $$

which results in a new Jacobian for end-effector velocity, denoted ${}^{E}J(q)$.

# Jacobian in the End-effector Coordinate Frame

- In the ***Toolbox*** this Jacobian is computed by the method `jacobe` and for the UR5 robot at the pose used above is:

*>>> ur5.jacobe(ur5.q1)*

```
array([[ -0.4745,  0,        0,        0,        0.0823, 0],
       [ 0.02685, -0.3303,  0.09465, 0.09465, 0.0823, 0],
       [ 0,        0.4746,  0.4745,  0.0823,  0,      0.0823],
       [ 0,       -1,      -1,      -1,        0,      0],
       [ 0,        0,        0,        0,        0,     -1],
       [-1,        0,        0,        0,        1,      0]])
```

# Ch. 8.3

## Jacobian Condition and Manipulability

# Jacobian Condition and Manipulability

- We have discussed how the ***Jacobian matrix*** maps joint rates to end-effector Cartesian velocity;
- However, the ***inverse problem*** has strong practical use – what joint velocities are ***needed*** to achieve a required end-effector Cartesian velocity?

# Jacobian Condition and Manipulability

- We have discussed how the ***Jacobian matrix*** maps joint rates to end-effector Cartesian velocity;
- However, the ***inverse problem*** has strong practical use – what joint velocities are ***needed*** to achieve a required end-effector Cartesian velocity?
- We can invert the ***differential kinematics*** equation (8.2) and write:

$$\dot{\boldsymbol{q}} = \mathbf{J}^{-1}(\boldsymbol{q})\boldsymbol{v} \tag{8.4}$$

provided that $\boldsymbol{J}$ is ***square*** and ***nonsingular*** (i.e., $\det(J) \neq 0$);

# Jacobian Singularities

- A ***robot configuratio***n $q$ at which $\det(\boldsymbol{J}(q)) = 0$ is described as singular or degenerate.
- Singularities occur when the robot is at ***maximum reach*** or when one or more axes become ***aligned*** resulting in the loss of degrees of freedom – the gimbal lock problem again.

# Jacobian Singularities

- Singularities can be classified into:

  - ***Boundary singularities***: that occur when the manipulator is either outstretched or retracted;

  It is also known as arm singularities…

# Jacobian Singularities

- Singularities can be classified into:

  - ❑ ***Boundary singularities***: that occur when the manipulator is either outstretched or retracted;

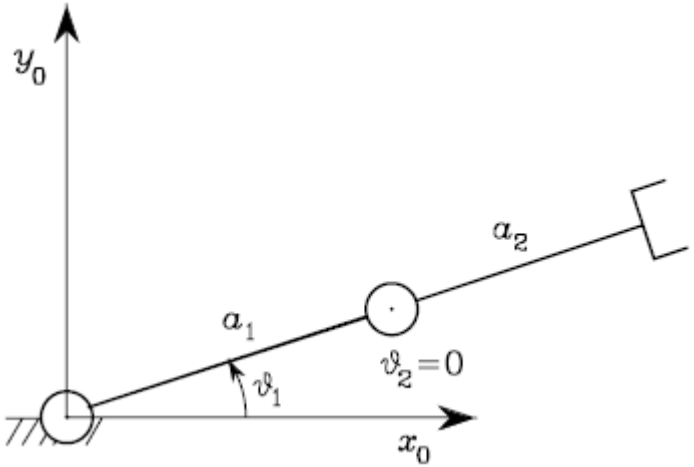
  It is also known as arm singularities…

  - ❑ ***Internal singularities***: that occur inside the reachable workspace and are generally caused by the alignment of two or more axes of motion;

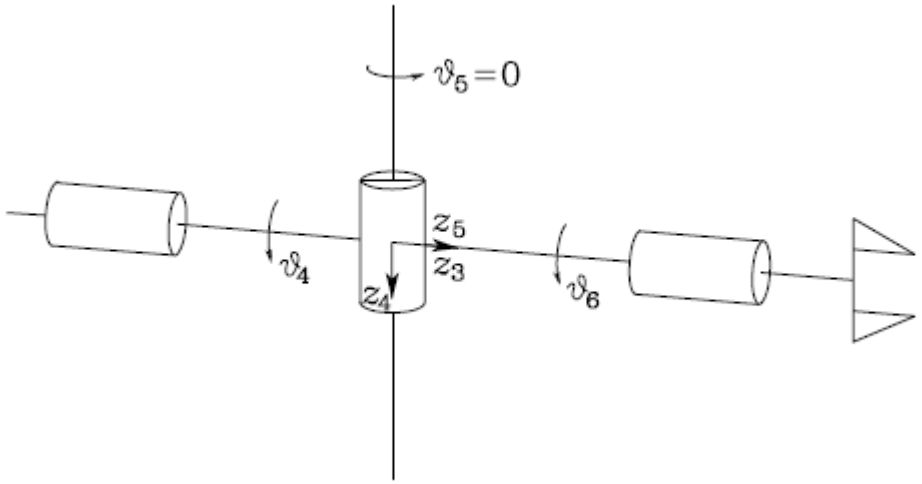
  It is also known as wrist singularities…

# Jacobian Singularities

- Arm and wrist singularities:




Two-link planar arm at a boundary singularity [2].




Spherical wrist at a singularity [2].

# Jacobian Singularities

- A ***robot configuratio***n $q$ at which $\det(\boldsymbol{J}(q)) = 0$ is described as singular or degenerate.
- Singularities occur when the robot is at ***maximum reach*** or when one or more axes become ***aligned*** resulting in the loss of degrees of freedom – the gimbal lock problem again.
- For example, at the UR5 robot at its *zero* configuration has a Jacobian

```
>>> J = ur5.jacob0(ur5.qz)
```

which is singular…

# Jacobian Singularities

```
J = array([[-0.1915, -0.09465, -0.09465, -0.09465, 0.0823, 0],
           [ 0.8996,  0,        0,        0,      -0.0823, 0],
           [ 0,      -0.8996,  -0.4746,  -0.0823,  0,     -0.0823],
           [ 0,       0,        0,        0,       0,      0],
           [ 0,       1,        1,        1,       0,      1],
           [ 1,       0,        0,        0,      -1,      0]])
```

since:

```
>>> np.linalg.det(J)
0
```

# Jacobian Singularities

- We can also see that the ***Jacobian rank*** is only

```
>>> np.linalg.matrix_rank(J)
5
```

- The function `jsingu` performs this analysis automatically, for example

```
>>> jsingu(J)
column 3 = - 0.923 column_1 + 1.92 column_2
```

indicating that velocity of $q_3$ can be expressed ***completely*** in terms of the velocities of $q_1$ and $q_2$.

- This means that joint 3 is **not needed**, and the robot has only 5 effective joints

# Jacobian Singularities

- However, if the robot is ***close*** to, but ***NOT*** actually at, a singularity we encounter problems where some Cartesian end-effector velocities require ***very high joint rates*** – at the singularity those rates will go to infinity;

# Jacobian Singularities

- However, if the robot is ***close*** to, but ***NOT*** actually at, a singularity we encounter problems where some Cartesian end-effector velocities require ***very high joint rates*** – at the singularity those rates will go to infinity;
- We can illustrate this by choosing a configuration ***slightly away*** from $q_z$ which we just showed was singular;

# Jacobian Singularities

- However, if the robot is ***close*** to, but ***NOT*** actually at, a singularity we encounter problems where some Cartesian end-effector velocities require ***very high joint rates*** – at the singularity those rates will go to infinity;
- We can illustrate this by choosing a configuration ***slightly away*** from $q_z$ which we just showed was singular;
- We set all joints to a small but nonzero value of ***5 deg***

```
>>> qns = np.full((6,), np.deg2rad(5))
```

```
array([ 0.08727, 0.08727, 0.08727, 0.08727, 0.08727, 0.08727])
```

# Jacobian Singularities

…and the ***Jacobian*** matrix is now

*>>> J = ur5.jacob0(qns);*

- To achieve relatively ***slow*** end-effector motion of $0.1\ ms^{-1}$ in the $z$-direction requires

*>>> qd = np.linalg.inv(J) @ [0, 0, 0, 0.1, 0, 0]*

```
array([-0.003797, -2.773, 5.764, -4.088, -0.02945, 1.093])
```

# Jacobian Singularities

…a very ***high-speed*** motion of many joints – the velocity of joint 2 is nearly 1 revolution /s, or **50 times** greater than the required task space rotation rate!

# Jacobian Singularities

…a very ***high-speed*** motion of many joints – the velocity of joint 2 is nearly 1 revolution /s, or **50 times** greater than the required task space rotation rate!

- The reason is that although the robot is ***NO*** longer at a singularity, the **determinant** of the Jacobian is still very small:

```
>>> np.linalg.det(J)
-0.0009943
```

- The closer we get to the singularity the higher the joint rates will be – at the **singularity** those rates will **go to infinity**.

# Jacobian Singularities

- Alternatively, we can say that its ***condition number*** is very high

```
>>> np.linalg.cond(J)
232.1
```

and the ***Jacobian*** is poorly conditioned.

# Manipulability

- Consider the set of ***generalized joint velocities*** with a unit norm

$$\dot{\boldsymbol{q}}^T \dot{\boldsymbol{q}} = 1 \tag{17}$$

which lie on the surface of a hypersphere in the $N$-dimensional joint velocity space.

# Manipulability

- Consider the set of ***generalized joint velocities*** with a unit norm

$$\dot{\boldsymbol{q}}^T \dot{\boldsymbol{q}} = 1 \qquad (*)$$

which lie on the surface of a hypersphere in the $N$-dimensional joint velocity space.

- Substituting Eq. (8.4) ($\dot{q} = J(q)\, v$) into (*) we get

$$\boldsymbol{v}^\top \left( \mathbf{J}(\boldsymbol{q}) \mathbf{J}^\top(\boldsymbol{q}) \right)^{-1} \boldsymbol{v} = 1 \qquad (8.7)$$

which is the equation of points ***v*** on the surface of a **hyper-*ellipsoid*** within the $\dim \mathcal{T}$-dimensional task-velocity space;

# Manipulability

- If this ***ellipsoid*** is close to spherical, that is, its ***radii*** are of the same order of magnitude then all is doing well – the end-effector ***CAN*** achieve arbitrary Cartesian velocity;

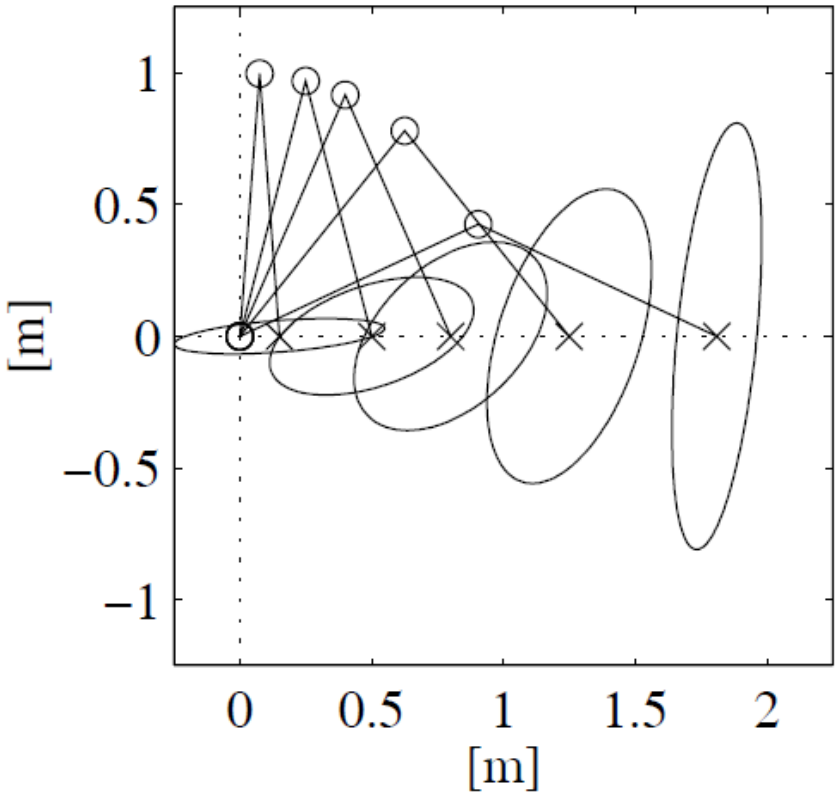



Fig.: Velocity manipulability ellipses for a ***2-link planar arm*** in different postures [2].

# Manipulability

- If this ***ellipsoid*** is close to spherical, that is, its ***radii*** are of the same order of magnitude then all is doing well – the end-effector ***CAN*** achieve arbitrary Cartesian velocity;
- However, if ***one*** or ***more*** radii are very small this indicates that the end-effector ***CANNOT*** achieve velocity in the directions corresponding to those small radii.

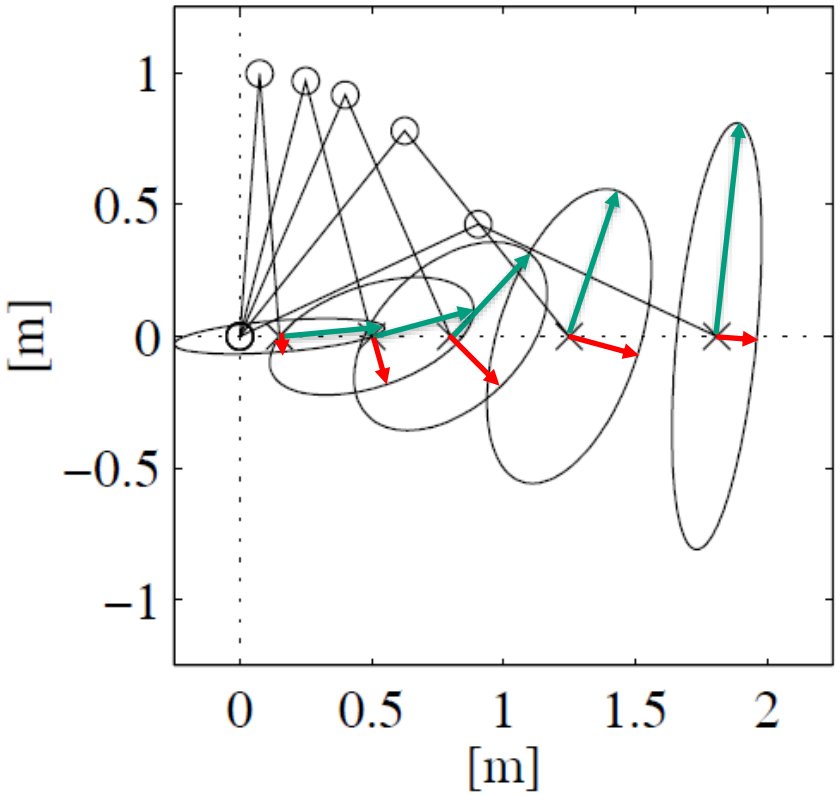



Fig.: Velocity manipulability ellipses for a ***2-link planar arm*** in different postures [2].

# Manipulability

- The ***size and shape*** of the ellipsoid describes how ***well-conditioned*** the manipulator is for making certain motions;
- Manipulability is a ***scalar measure*** that describes ***how*** spherical the ellipsoid is, for instance the ***ratio*** of the smallest to the largest radius;
- The Toolbox method manipulability computes ***Yoshikawa's manipulability*** measure as:

$$m(\boldsymbol{q}) = \sqrt{\det\left(\mathbf{J}(\boldsymbol{q})\mathbf{J}^\top(\boldsymbol{q})\right)} \in \mathbb{R} \tag{8.8}$$

which is proportional to the ***volume*** of the ellipsoid.

# Manipulability

- For example,

```
>>> ur5.manipulability(ur5.qz)
0
```

indicates a ***singularity***. Compared to:

```
>>> ur5.manipulability(ur5.q1)
0.06539
```

# Manipulability

- The method can be called with axes parameter ("trans", "rot" or "both")

```
>>> ur5.manipulability(ur5.qz, axes="both")

Manipulability: translation 0.00017794, rotation 0

(0.105, 2.449e-16)
```

Rotational manipulability

# Manipulability

- The method can be called with axes parameter ("trans", "rot" or "both")

```
>>> ur5.manipulability(ur5.qz, axes="both")
```

```
(0.105, 2.449e-16)
```

Rotational manipulability

- In practice we find that the seemingly large workspace of a robot is greatly reduced by joint limits, self collision, singularities and regions of low manipulability

# Wrapping up…

- ***Jacobians*** are an important concept in robotics, relating changes in one space to changes in another;
- The ***manipulator Jacobian*** which describes the relationship between the rate of change of ***joint coordinates*** and the ***spatial velocity*** of the end-effector expressed in either the world frame or the end-effector frame;
- The ***numerical properties*** of the Jacobian tell us about manipulability, that is how well the manipulator is able to move, or exert force, in different directions;
- At a ***singularity***, indicated by linear dependence between columns of the Jacobian, the robot is unable to move in ***certain directions***;

# Bibliography

[1] Corke, P., **Robotics, *Vision and Control: Fundamental Algorithms in Pythin***, *Springer International Publishing AG*, 3rd Ed., 2023.

[2] Siciliano, B., Sciavicco, Villani, L. & Oriolo, G., ***Robotics: Modeling, Planning and Control***, *Springer-Verlag London Limited,* 2nd Ed., 2009.

Thank you for your attention !